CANTINA

LI.FI OIF Contracts

Security Review

Cantina Managed review by:

Gerard Persoon, Lead Security Researcher
Kaden, Security Researcher

July 9, 2025

Contents

1 Introduction **3**
1.1 About Cantina . . . 3
1.2 Disclaimer . . . 3
1.3 Risk assessment . . . 3
1.3.1 Severity Classification . . . 3

2 Security Review Summary **4**

3 Findings **5**
3.1 High Risk . . . 5
3.1.1 Lock on fills and claim can be circumvented . . . 5
3.2 Medium Risk . . . 5
3.2.1 Deleting claimed orders allows for race condition . . . 5
3.2.2 Order ID hash collision possible . . . 6
3.2.3 Repeated outputs can be filled with one filling . . . 8
3.2.4 Function `_processMessage()` doesn't check the event topic . . . 8
3.2.5 `CROSS_L2_PROVER` contract is not verified on Etherscan . . . 8
3.3 Low Risk . . . 9
3.3.1 No explict check on `order.originChainId` in function `finalise()` . . . 9
3.3.2 After giving up ownership, fees could be lost . . . 9
3.3.3 Missing check in `InputSettlerCompactLIFI::_resolveLock()` . . . 9
3.3.4 Timestamp sometimes truncated in `_verify()` . . . 9
3.3.5 No `deadline` parameter in `BitcoinOracle.dispute` . . . 10
3.4 Gas Optimization . . . 10
3.4.1 `memory-safe` not used everywhere . . . 10
3.4.2 Functions `hashOutputs()` and `hashOutputsM()` can be optimized . . . 10
3.4.3 No calldata version of `encodeFillDescription()` . . . 11
3.4.4 Function `_isPayloadValid()` can be optimized . . . 11
3.4.5 Boolean `&&` calculations can be optimized . . . 11
3.4.6 Similar functions `verify()`/`_verify()` and `optimisticallyVerify()` emit different values . . . 12
3.4.7 Emit can use already present variables . . . 12
3.4.8 Efficient bit sanitization . . . 12
3.4.9 Part of check in `_resolveClaimed()` is always true . . . 13
3.4.10 Function `dispute()` reads value of `claimedOrder.claimTimestamp` twice . . . 13
3.4.11 Code duplication `optimisticallyVerify()` and `_resolveClaimed()` / `_verify()` . . . 13
3.4.12 Optimization in `optimisticallyVerify()` . . . 13
3.4.13 Function `optimisticallyVerify()` reads value of `claimedOrder.solver` twice. . . . 14
3.5 Informational . . . 14
3.5.1 Typos in comments . . . 14
3.5.2 Action for `destination == byte32(0)` unclear . . . 14
3.5.3 Comment in `setGovernanceFee()` is incorrect . . . 15
3.5.4 Duplicate import in `BaseInputSettler` . . . 15
3.5.5 Function `_proofPayloadHash()` could be used in more places . . . 15
3.5.6 Behaviour for single solver unclear . . . 15
3.5.7 Signature for `AllowOpen` has no time limit . . . 16
3.5.8 Old name `filler` still used . . . 16
3.5.9 Comment for function `hashOrderPurchase()` is incorrect . . . 16
3.5.10 The `nonce` of the `StandardOrder` is not obvious . . . 16
3.5.11 The string `MandateOutput(...` is repeated . . . 17
3.5.12 Order-validation documentation is incomplete . . . 17
3.5.13 Risks for callback functions . . . 17
3.5.14 Emit in function `_fill()` could be expanded . . . 18
3.5.15 Function `fillOrderOutputs()` reverts when `outputs[]` is empty . . . 18
3.5.16 Typecast to `bytes32()` done twice . . . 18
3.5.17 `encodeMessage()` and `decodeMessage()` not symmetric . . . 18
3.5.18 Incorrect comment in function `_getProofPeriod()` . . . 19
3.5.19 `BitcoinOracle::_outputIdentifier()` could use `MandateOutputEncoding::getMandateOutputHash()` . . . 19

3.5.20 Code duplication of `_isPayloadValid()` and `arePayloadsValid()` 19
3.5.21 Several functions write to `_attestations[][][][]` 19
3.5.22 `uint32` timestamps will fail beyond the year 2106 20
3.5.23 Incorrect `RegisterIntentLib` NatSpec comment 20
3.5.24 Order of checks in function `dispute()` not logical 20
3.5.25 `DISPUTE_PERIOD` can be shorter than the number of confirmations requested 21
3.5.26 Only function for one direction of mapping in `ChainMap` 21

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

LI.FI is a cross-chain bridge aggregation protocol that supports any-2-any swaps by aggregating bridges and connecting them to DEX aggregators.

From Jun 12th to Jun 23rd the Cantina team conducted a review of oif-contracts on commit hash ecebf5cc. The team identified a total of **50** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	5	3	2
Low Risk	5	4	1
Gas Optimizations	13	7	6
Informational	26	15	11
Total	**50**	**30**	**20**

3 Findings

3.1 High Risk

3.1.1 Lock on fills and claim can be circumvented

Severity: High Risk

Context: MandateOutputEncodingLib.sol#L117-L142, MandateOutputEncodingLib.sol#L147-L163, BitcoinOracle.sol#L557-L561, BaseOutputSettler.sol#L65-L88, OutputSettlerCoin.sol#L119-L126

Description: In `BaseOutputSettler::_fill()` the following happens:

- `MandateOutputEncodingLib.getMandateOutputHash(output)` uses `output.oracle` and is used as a re-entry check. `MandateOutputEncodingLib.encodeFillDescription()` doesn't use `output.oracle` and is used as the index for `_attestations[][][][]`. This means the same value for `_attestations[][][][]` can be set with different values for `output.oracle`. So the reentrancy check doesn't work as expected.
- Neither `OutputSettlerCoin::_fill()` nor `BaseOutputSettler::_fill()` check `output.oracle`, although the `output.settler` is checked.
- This allows filling the same order multiple times. A second `solver` could try to out compete the first `solver` by quickly calling `fill()` and `finalise()`.
- A side effect is that one of the two solvers won't be paid back, so their funds are lost.
- In `BitcoinOracle::claim()` the same situation is present:
  - `outputId` doesn't contain the `output.oracle` so with a different value for `output.oracle` a different `claim()` can be done, which can still be filled and the inputs can be claimed.

This can be exploited by the order creator by filling and finalizing their own order before a legitimate solver can finalize their completed fill. Consider, for example, the following scenario:

- Attacker creates order to be filled via `BitcoinOracle`.
- Attacker calls `claim` with themselves as `solver` and an invalid `output.oracle`.
- Victim calls `claim` with valid `output.oracle`.
  - The `claim` succeeds, incorrectly indicating to the victim that they are the first to claim.
- Attacker fills the order on bitcoin.
- Victim fills the order on bitcoin.
- Attacker verifies and finalizes order before victim, receiving the input and output of the order.
- Victim cannot finalize their order but has already transferred the bitcoin to the attacker.

Since the attacker is filling their own order, i.e. sending funds to themselves, they don't incur any risk, allowing them to repeat it indefinitely.

Recommendation: Consider also checking `output.oracle` in both `BaseOutputSettler::_fill()` and `BitcoinOracle::claim()`. This is also relevant if future versions of an output settler would support multiple oracles.

For completeness also check `output.oracle` in `BitcoinOracle::optimisticallyVerify()`.

When updating `_attestations[][][][]` (in all parts of the code) also consider checking the value isn't already set.

LI.FI: Fixed in PR 12. Slightly different implementation than recommended. However, this implements a significant optimisation.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Deleting claimed orders allows for race condition

Severity: Medium Risk

Context: BitcoinOracle.sol#L511, BitcoinOracle.sol#L531-L537, BitcoinOracle.sol#L612, BitcoinOracle.sol#L631-L636

Description: In `BitcoinOracle._resolveClaimed`, in the case that an order has been successfully filled before being disputed, we clear the `_claimedOrder[orderId][outputId]`:

```
delete claimedOrder.solver;
delete claimedOrder.multiplier;
delete claimedOrder.claimTimestamp;
delete claimedOrder.sponsor;
delete claimedOrder.disputer;
delete claimedOrder.disputeTimestamp;
```

One side effect of this logic is that we now technically allow a second claim to be created for the same order since we validate that an order hasn't already been claimed by checking the `claimedOrder.solver`:

```
ClaimedOrder storage claimedOrder = _claimedOrder[orderId][outputId];
if (claimedOrder.solver != bytes32(0)) revert AlreadyClaimed(claimedOrder.solver);
```

The same resource lock cannot be claimed twice, so as long as the first claim is finalized first, the second claim poses no threat. However, it's worth noting that the sponsor of the resource lock controls some relevant parameters:

- The multiplier, via `output.context`, can be as low as 1 wei (effectively free collateral).
- The `numConfirmations`, via `output.token`, can be as low as 0 (speeding up the claim verification process).

As a result, an attacker can create orders and immediately claim them after they are verified by the solver, racing the solver to finalize the order and retrieve the inputs first, receiving the output from the solver while paying the inputs to themself. This is relatively unlikely as there will still be a relatively significant delay between claiming and verifying, but since the cost to attempt the attack is cheap, it can be easily repeated. In case a solver is inefficient or is delayed due to some offchain logic bug, theft could occur.

Note: the same issue is present with function `optimisticallyVerify()`.

Recommendation: It's important that we maintain some indicator that an order has been claimed, even after the claim has been verified. There are two potential options for accomplishing this:

- Maintain `claimedOrder.solver` after verifying, making sure to implement a different form of reentrancy protection, or.
- Add a boolean to `ClaimedOrder` indicating whether it has been claimed and don't clear it during verification.

> Note:
> The same remediation must also be implemented in the `optimisticallyVerify` function. Additionally, it's important that we still clear the `claimedOrder` entirely in `finaliseDispute`.

LI.FI: Fixed in PR 15. Implementation by using `claimedOrder.solver`.

Cantina Managed: Fix verified.

3.2.2 Order ID hash collision possible

Severity: Medium Risk

Context: StandardOrderType.sol#L34-L50

Description: In `StandardOrderType.orderIdentifier`, we compute the resulting order ID as the hash of the packed encoding of the `order`:

```
function orderIdentifier(
    StandardOrder calldata order
) internal view returns (bytes32) {
    return keccak256(
        abi.encodePacked(
            block.chainid,
            address(this),
            order.user,
            order.nonce,
            order.expires,
            order.fillDeadline,
            order.localOracle,
            order.inputs,
            abi.encode(order.outputs)
        )
    );
}
```

The potential risk here lies in the fact that `order.inputs` is a dynamically typed value (dynamic length array), and by using packed encoding, we exclude the length of the array. As a result, there is no indicator as to when the `order.inputs` ends and the next value starts, which can lead to a hash collision if an order is maliciously constructed. For example, we can craft an order with additional inputs which match the first `order.outputs` element to have the same resulting order ID: Valid order with one input and multiple outputs:

```
...
// Inputs
Inputs[0][0]
inputs[1][0]

// Outputs
Encoded output offset
encoded output length
outputs[0].oracle
outputs[0].settler
...
Outputs[1].oracle
Outputs[1].settler
...
```

Invalid order which replaces the first output with several inputs, resulting in the same order ID:

```
...
// Inputs
Inputs[0][0]
inputs[1][0]
Inputs[0][1] = Encoded output offset
inputs[1][1] = encoded output length
Inputs[0][2] = outputs[0].oracle
inputs[1][2] = outputs[0].settler
...

// Outputs (skips first index)
Outputs[1].oracle
Outputs[1].settler
...
```

In this case, it's unlikely to result in an exploit due to the fact that the inputs and outputs will be invalid, likely causing multiple possible reverts. Regardless, it's recommended to remediate this to prevent any possible hash collisions.

Recommendation: `abi.encode` the `order.inputs` array before including it in the packed encoding:

```
  function orderIdentifier(
      StandardOrder calldata order
  ) internal view returns (bytes32) {
      return keccak256(
          abi.encodePacked(
              block.chainid,
              address(this),
              order.user,
              order.nonce,
              order.expires,
              order.fillDeadline,
              order.localOracle,
-             order.inputs,
+             abi.encode(order.inputs),
              abi.encode(order.outputs)
          )
      );
  }
```

LI.FI: Fixed in PR 14. Different implementation than recommended.

Cantina Managed: Fix verified.

3.2.3 Repeated outputs can be filled with one filling

Severity: Medium Risk

Context: BaseOutputSettler.sol#L56-L58, BaseOutputSettler.sol#L65

Description: Repeated outputs can be filled with one filling, as documented in a comment on function `BaseOutputSettler::fill()`. As this is not obvious, some mistakes with is are bound to be made. This means the sponser would loose funds as they don't get the fills they expect.

Recommendation: Consider adding a sequence number to the outputs to prevent the possibility of having the same outputs.

LI.FI: We won't fix this. The `orderId` provides this protection. This would increase the chance of an issue happening on submission and increase the chance of loss of funds.

Cantina Managed: Acknowledged.

3.2.4 Function `_processMessage()` doesn't check the event topic

Severity: Medium Risk

Context: PolymerOracle.sol#L40-L47

Description: Function `_processMessage()` doesn't check the event topic so the rest of the code could match with any event. Although its unlikely that another event can be decoded, it is a lot safer to check the event topic.

Recommendation: Consider checking `topics[0:32] == OutputFilled.selector`.

LI.FI: Fixed in PR 13.

Cantina Managed: Fix verified.

3.2.5 `CROSS_L2_PROVER` contract is not verified on Etherscan

Severity: Medium Risk

Context: PolymerOracle.sol#L40-L44

Description: The `CROSS_L2_PROVER` contract isn't verified on etherscan, see below. It should be easy to verify because the source is public. Without a verified contract is difficult to rely on the oracle and this might prevent solvers to use this oracle. An unverified contract could potentially hide a vulnerability.

Recommendation: Request the Polymer project to verify the contracts.

LI.FI: Acknowledged.

Cantina Managed: Acknowledged.

3.3 Low Risk

3.3.1 No explict check on `order.originChainId` in function `finalise()`

Severity: Low Risk

Context: InputSettlerCompactLIFI.sol#L84-L94, InputSettlerCompactLIFI.sol#L119, InputSettlerCompact.sol#L66-L70, InputSettlerCompact.sol#L177, InputSettlerCompact.sol#L213, StandardOrderType.sol#L34-L39

Description: There is no explicit check that `order.originChainId == block.chainid` in function `finalise()`. Function `StandardOrderType::orderIdentifier()` just uses `block.chainid`. This could allow calling `finalise()` on the wrong chain. Other checks will prevent retrieving funds, but it is safer to check this.

Recommendation: Consider checking `order.originChainId == block.chainid` in functions `finalise()` and `finaliseWithSignature()` of both `InputSettlerCompactLIFI.sol` and `InputSettlerCompact.sol`.

LI.FI: Fixed in oif-contracts PR 17 and catalyst-intent PR 105.

Cantina Managed: Fix verified.

3.3.2 After giving up ownership, fees could be lost

Severity: Low Risk

Context: InputSettlerCompactLIFI.sol#L175

Description: If, in the future, the ownership of `InputSettlerCompactLIFI` would be given up via `renounceOwnership()`, then the collected `fee` tokens will be send to `address(0)` and thus lost.

Recommendation: Consider detecting the `owner` is set to `address(0)` and in that case handle the `fee` differently, for example set the `fee` to 0.

LI.FI: Fixed in PR 104.

Cantina Managed: Fix verified.

3.3.3 Missing check in `InputSettlerCompactLIFI::_resolveLock()`

Severity: Low Risk

Context: InputSettlerCompactLIFI.sol#L147, InputSettlerCompact.sol#L250, InputSettlerCompact.sol#L277-L278

Description: Function `InputSettlerCompact.sol::_resolveLock()` revert when `user == address(this)`, however function `InputSettlerCompactLIFI::_resolveLock()`, which overrides `InputSettlerCompact.sol::_resolveLock()`, doesn't have this check.

Recommendation: Consider also adding this check to `InputSettlerCompactLIFI::_resolveLock()`.

LI.FI: Fixed in PR 103.

Cantina Managed: Fix verified.

3.3.4 Timestamp sometimes truncated in `_verify()`

Severity: Low Risk

Context: BitcoinOracle.sol#L381-L411

Description: Function `_verify()` uses `timestamp` in two different ways: first untruncated and the second truncated to `uint32()`. This could be abused to circumvent the first check by suppying a `timestamp` that is a multiple of `type(uint32.max) + 1`. The truncated to `uint32()` version of this is 0, which can circumvent checks in `_resolveClaimed()`. Although other checks prevent attacks, its safer to remove this attack path.

Recommendation: Considering do the following at the start of function `_verify()`:

```
timestamp = uint32(timestamp);
```

LI.FI: Fixed in PR 16.

Cantina Managed: Fix verified.

3.3.5 No `deadline` parameter in `BitcoinOracle.dispute`

Severity: Low Risk

Context: BitcoinOracle.sol#L585

Description: To successfully dispute a claim in the `BitcoinOracle` , `dispute` must be called before the bitcoin transaction is filled. In case a `dispute` transaction is submitted before the bitcoin transaction is filled, but only processed after the bitcoin transaction is filled, the dispute will fail and the disputer will lose their provided collateral.

In practice, this is relatively easy to avoid since there's ~10 minutes between bitcoin blocks, so there is a sizeable gap in which the bitcoin transaction cannot be filled. Regardless, it can be difficult to ensure a transaction is processed in time without implementing a `deadline` parameter beyond which the call reverts.

Recommendation: Implement a `deadline` parameter in `BitcoinOracle.dispute`, beyond which the call will revert.

LI.FI: Acknowledged.

Cantina Managed: Acknowledged.

3.4 Gas Optimization

3.4.1 `memory-safe` not used everywhere

Severity: Gas Optimization

Context: MandateOutputType.sol#L61, MandateOutputType.sol#L96, BytesLib.sol#L13

Description: Most instance of `assembly` alse have `memory-safe`, however there are a few exceptions.

Recommendation: Consider using `memory-safe` everywhere.

```
- assembly {
+ assembly ("memory-safe") {
```

LI.FI: Fixed in PR 33.

Cantina Managed: Fix verified.

3.4.2 Functions `hashOutputs()` and `hashOutputsM()` can be optimized

Severity: Gas Optimization

Context: MandateOutputType.sol#L53-L67, MandateOutputType.sol#L88-L102

Description: The functions `hashOutputs()` and `hashOutputsM()` can be optimized to save some gas.

Recommendation: Consider optimizing the code in the following way:

```
+  uint p;
+  assembly {
+     p := add(currentHash,0x20)
+  }
   for (uint256 i = 0; i < outputs.length; ++i) {
       bytes32 outputHash = hashOutput(outputs[i]);
       assembly {
-          mstore(add(add(currentHash, 0x20), mul(i, 0x20)), outputHash)
+          mstore(p , outputHash)
+          p := add(p, 0x20)
        }
   }
```

LI.FI: Fixed in PR 32.

Cantina Managed: Fix verified.

3.4.3 No calldata version of `encodeFillDescription()`

Severity: Gas Optimization

Context: MandateOutputEncodingLib.sol#L117-L126, MandateOutputEncodingLib.sol#L147-L163

Description: The is no `calldata` version of `encodeFillDescription( < 8 parameters> )`, so it is not much use to have a `calldata` version of `encodeFillDescription( < 4 parameters> )`.

Recommendation: Consider also adding a `calldata` version of `encodeFillDescription( < 8 parameters> )` and using that from the `calldata` version of `encodeFillDescription( < 4 parameters> )`.

LI.FI: Fixed in PR 31.

Cantina Managed: Fix verified.

3.4.4 Function `_isPayloadValid()` can be optimized

Severity: Gas Optimization

Context: BaseOutputSettler.sol#L176-L182

Description: The first three parameters to `_attestations[] [] [] []` are always the same, so can access to the array can be made more efficient.

Recommendation: Consider changing the code to somehting like the following:

```
function _isPayloadValid(mapping(bytes32 dataHash => bool) storage p, bytes32 payloadHash) /*...*/ {
    return p[payloadHash];
}
mapping(bytes32 dataHash => bool) storage p =
    _attestations[block.chainid]
        [bytes32(uint256(uint160(address(this))))]
        [bytes32(uint256(uint160(address(this))))];

function arePayloadsValid(...) ... {
    // ...
    accumulator = accumulator && _isPayloadValid2(p,bytes32(i));
    // ...
}
```

LI.FI: Acknowledged. This code has been removed.

Cantina Managed: Acknowledged.

3.4.5 Boolean `&&` calculations can be optimized

Severity: Gas Optimization

Context: BaseOracle.sol#L68-L104, BitcoinOracle.sol#L299-L308, BaseOutputSettler.sol#L187-L196, BaseOutputSettler.sol#L187-L197

Description: Several function use a loop where `bool` variable is `&&`-id with result of the cycle. As `bool` variables are stored as a `byte` and the EVM works with 32 byte words, this not efficient.

Recommendation: Consider using words instead of `bool` plus `&` (`and`) instead of `&&`:

```
uint accumulator = 1;

for (/*...*/) {
    assembly ("memory-safe") {
        // b is either 0 or 1
        accumulator := and(accumulator,b)
    }
}
assembly ("memory-safe") {
    accumulator := and(accumulator,1)  // strip any high order bits
}
```

LI.FI: Fixed in PR 30.

Cantina Managed: Fix verified.

3.4.6 Similar functions `verify()`/`_verify()` and `optimisticallyVerify()` emit different values

Severity: Gas Optimization

Context: BitcoinOracle.sol#L381, BitcoinOracle.sol#L410, BitcoinOracle.sol#L463, BitcoinOracle.sol#L612, BitcoinOracle.sol#L642

Description: The similar functions `verify()`/`_verify()` and `optimisticallyVerify()` emit different values.

Recommendation: In function `_verify()` consider also emitting `_outputIdentifier(output)`.

LI.FI: Won't fix. Each function emits the minimum it can for its necessary context.

Cantina Managed: Acknowledged.

3.4.7 Emit can use already present variables

Severity: Gas Optimization

Context: BitcoinOracle.sol#L585-L586, BitcoinOracle.sol#L604, BitcoinOracle.sol#L612-L613, BitcoinOracle.sol#L642, BitcoinOracle.sol#L650-L651, BitcoinOracle.sol#L680

Description: The emits in functions `finaliseDispute()`, `OutputOptimisticallyVerified` and `OutputDisputed` calculate `_outputIdentifier(output)`. However this value is already stored in `outputId` and could be used.

Recommendation: Consider changing the emits to:

```
- emit ...(orderId, _outputIdentifier(output));
+ emit ...(orderId, outputId);
```

LI.FI: Fixed in PR 29.

Cantina Managed: Fix verified.

3.4.8 Efficient bit sanitization

Severity: Gas Optimization

Context: OutputSettlerCoin.sol#L100-L102

Description: In the `orderType == 0xe1` case of `OutputSettlerCoin._orderType`, we retrieve the `startTime` and `stopTime` from the `fulfillmentContext`, cleaning the leftmost bits in the process:

```
// Clean the leftmost bytes: (32-4)*8 = 224
startTime := shr(224, shl(224, calldataload(add(fulfillmentContext.offset, 5))))
stopTime := shr(224, shl(224, calldataload(add(fulfillmentContext.offset, 9))))
```

Here we `calldataload` the values from `fulfillmentContext` as the rightmost bytes and clear the leftmost bytes, requiring two bit shifts to end up with a little endian value. We can shave off a little bit of gas by instead retrieving them as the leftmost bytes and only shifting right.

Recommendation: Adjust the offset provided to `calldataload` to retrieve the `startTime` and `stopTime` as the leftmost bytes and only shifting right, e.g.:

```
// Clean the rightmost bytes: (32-4)*8 = 224
startTime := shr(224, calldataload(add(fulfillmentContext.offset, 33))))
stopTime := shr(224, calldataload(add(fulfillmentContext.offset, 37))))
```

LI.FI: Won't fix. We are concerned about there being enough calldata.

Cantina Managed: Acknowledged.

3.4.9 Part of check in `_resolveClaimed()` is always true

Severity: Gas Optimization

Context: BitcoinOracle.sol#L511-L528, BitcoinOracle.sol#L557-L570

Description: Once the code arrives at L528: `if (sponsor != address(0) && (...) || ... ))`, then `solver` must be set (otherwise it would have reverted). `claimedOrder.sponsor` is set at the same time as `claimedOrder.solver`, in function `claim()`. So `sponsor` must also be set and there is no need to check for it again.

Recommendation: Consider changing the check to:

```
- if (sponsor != address(0) && (disputer == address(0) || fillTimestamp <= disputeTimestamp)) {
+ if (disputer == address(0) || fillTimestamp <= disputeTimestamp) {
```

LI.FI: Won't fix. No longer true due to other changes.

Cantina Managed: Acknowledged.

3.4.10 Function `dispute()` reads value of `claimedOrder.claimTimestamp` twice

Severity: Gas Optimization

Context: BitcoinOracle.sol#L585-L596

Description: Function `dispute()` reads the value of `claimedOrder.claimTimestamp` twice. Some gas could be saved by caching this value.

Recommendation: Consider caching `claimedOrder.claimTimestamp`.

LI.FI: Acknowledged.

Cantina Managed: Acknowledged.

3.4.11 Code duplication `optimisticallyVerify()` and `_resolveClaimed()` / `_verify()`

Severity: Gas Optimization

Context: BitcoinOracle.sol#L612

Description: There is quite a lot of Code duplication `optimisticallyVerify()` and `_resolveClaimed()` / `_verify()`. This makes the code more difficult to maintain and more expensive to deploy.

Recommendation: Consider refactoring the code to reduce duplication.

LI.FI: Acknowledged.

Cantina Managed: Acknowledged.

3.4.12 Optimization in `optimisticallyVerify()`

Severity: Gas Optimization

Context: BitcoinOracle.sol#L612-L619

Description: The variable `disputed` is used only once, so there is no need to store it.

Recommendation: Consider changing the code to:

```
- bool disputed = claimedOrder.disputer != address(0);
- if (disputed) revert Disputed();
+ if (claimedOrder.disputer != address(0)) revert Disputed()
```

LI.FI: Fixed in PR 28.

Cantina Managed: Fix verified.

3.4.13 Function `optimisticallyVerify()` reads value of `claimedOrder.solver` twice.

Severity: Gas Optimization

Context: BitcoinOracle.sol#L612-L621

Description: Function `optimisticallyVerify()` reads value of `claimedOrder.solver` twice. Some gas could be saved bystoring this value in `solver` earlier.

Recommendation: Consider changing the code to:

```
+  bytes32 solver = claimedOrder.solver;
-  if (claimedOrder.solver == bytes32(0)) revert NotClaimed();
+  if (solver == bytes32(0)) revert NotClaimed();
   // ...
-  bytes32 solver = claimedOrder.solver;
```

LI.FI: Fixed in PR 28.

Cantina Managed: Fix verified.

3.5 Informational

3.5.1 Typos in comments

Severity: Informational

Context: InputSettlerCompactLIFI.sol#L49, GovernanceFee.sol#L9, RegisterIntentLib.sol#L96, RegisterIntentLib.sol#L98, MessageEncodingLib.sol#L17, BitcoinOracle.sol#L29, BitcoinOracle.sol#L440, BitcoinOracle.sol#L485

Description: The code contains several typos in the comments.

Recommendation: Consider fixing the following typos:

- `InputSettlerCompactLIFI::broadcast()`: Valdiates → Validates.
- `RegisterIntentLib::depositAndRegisterIntentFor()`: registerers → registers.
- `RegisterIntentLib::depositAndRegisterIntentFor()`: or or → or.
- `MessageEncodingLib`: bytes3 → bytes.
- `GovernanceFee`: the the → the.
- `BitcoinOracle`: Confiramtions → Confirmations.
- `BitcoinOracle::verify()` (2 instances): Hheader → Header.

LI.FI: Fixed in catalyst-intent PR 110 and oif-contracts PR 27.

Cantina Managed: Fix verified.

3.5.2 Action for `destination == byte32(0)` unclear

Severity: Informational

Context: InputSettlerCompactLIFI.sol#L71-L92, InputSettlerCompact.sol#L166-L177

Description: A comment for function `finalise()` says:

> If destination is bytes32(0), the order owner will be used as the destination.

However the code reverts when `destination == bytes32(0)`.

Recommendation: Doublecheck which action should be taken and adapt the code or the comment.

LI.FI: Fixed in catalyst-intent PR 109 and oif-contracts PR 26.

Cantina Managed: Fix verified.

3.5.3 Comment in `setGovernanceFee()` is incorrect

Severity: Informational

Context: GovernanceFee.sol#L53-L56

Description: A comment at `setGovernanceFee()` says the fee update is applied immediately, however it is only applied after calling `applyGovernanceFee()`.

Recommendation: Consider changing the comment to something like:

```
- * @notice Sets a new governanceFee. Is immediately applied to orders initiated after this call.
+ * @notice Sets a new governanceFee. Is applied after a call to applyGovernanceFee
```

LI.FI: Fixed in PR 108.

Cantina Managed: Fix verified.

3.5.4 Duplicate import in `BaseInputSettler`

Severity: Informational

Context: BaseInputSettler.sol#L6-L7

Description: The contract BaseInputSettler imports `SignatureCheckerLib` twice, which means the second import is redundant.

Recommendation: Import `SignatureCheckerLib` only once.

LI.FI: Fixed in PR 25.

Cantina Managed: Fix verified.

3.5.5 Function `_proofPayloadHash()` could be used in more places

Severity: Informational

Context: InputSettlerCompact.sol#L80-L87, BitcoinOracle.sol#L404-L405, BitcoinOracle.sol#L623-L624, PolymerOracle.sol#L30-L38, BaseOutputSettler.sol#L84-L86

Description: The expression `keccak256(MandateOutputEncodingLib.encodeFillDescription())` is wrapped in `_proofPayloadHash()` in both the contracts `InputSettlerCompact` and `PolymerOracle`. However in `BaseOutputSettler` and `BitcoinOracle` the expression is used directly. This is inconsistent and makes the code more difficult to understand.

Recommendation: For consistency consider using the same pattern everywhere.

> Note: there are different versions of the expression and function: a `calldata` and a `memory` based version.

LI.FI: Acknowledged. We are unsure of what the best way to approach this is. So currently won't fix but in the future we may fix.

Cantina Managed: Acknowledged.

3.5.6 Behaviour for single solver unclear

Severity: Informational

Context: InputSettlerCompactLIFI.sol#L78-L84, InputSettlerCompactLIFI.sol#L103, InputSettlerCompact.sol#L94-L113, InputSettlerCompact.sol#L171-L177, InputSettlerCompact.sol#L207-L213

Description: A comment in function `InputSettlerCompactLIFI::finalise()` says:

> For single solver, pass an array with only one element

However `InputSettlerCompactLIFI::finalise()` calls `InputSettlerCompact::_validateFills()` which loops over the entire array `solvers[]` and will revert if only one `solver` is provided.

Recommendation: Check if the comment or the code is correct and adapt the comment and code. Also consider checking `solvers.length == MandateOutputs.length` in case more than one `solver` is provided.

LI.FI: Fixed in catalyst-intent PR 107 and oif-contracts PR 24.

Cantina Managed: Fix verified.

3.5.7 Signature for `AllowOpen` has no time limit

Severity: Informational

Context: BaseInputSettler.sol#L89-L96, InputSettlerCompact.sol#L213-L230, AllowOpenType.sol#L18-L20, AllowOpenType.sol#L29-L31

Description: The function `hashAllowOpen()` allows the signature for `AllowOpen` to be used indefintely. It might be useful to limit time when the signature is valid.

Recommendation: Consider adding a deadline to the hash.

LI.FI: Won't fix. These are short-lived because intents are short lived.

Cantina Managed: Acknowledged.

3.5.8 Old name `filler` still used

Severity: Informational

Context: MandateOutputType.sol#L29-L30, MandateOutputType.sol#L35-L42, StandardOrderType.sol#L53-L60, MandateOutputEncodingLib.sol#L15-L19

Description: The strings `MANDATE_OUTPUT_TYPE_STUB`, `BATCH_COMPACT_SUB_TYPES` and `CATALYST_WITNESS_TYPE` use the old name `filler` which has been replaced by `settler`. Similarly a comment in `MandateOutputEncodingLib` still uses `REMOTE_FILLER`. The order-validation documentation also contains `output.remoteFiller`.

Recommendation: Consider replacing all instances of `(remote)filler` with `settler`.

LI.FI: Fixed in oif-contracts PR 10 and catalyst-intent commit 27ce0972.

Cantina Managed: Fix verified.

3.5.9 Comment for function `hashOrderPurchase()` is incorrect

Severity: Informational

Context: OrderPurchaseType.sol#L28-L32

Description: The comment for function `hashOrderPurchase()` is incorrect.

Recommendation: Consider changing the comment to:

```
- @notice Hashes an AllowOpen struct.
+ @notice Hashes an OrderPurchase struct.
```

LI.FI: Fixed in PR 23.

Cantina Managed: Fix verified.

3.5.10 The `nonce` of the `StandardOrder` is not obvious

Severity: Informational

Context: StandardOrderType.sol#L6-L8

Description: The `nonce` of the `StandardOrder` is the `allocator nonce`, which is not obvious.

Recommendation: Consider adding a comment indicating its the `allocator nonce`.

LI.FI: Won't fix. The `standard order` will be used in implementations where the `nonce` may not be an `allocator nonce`. The order is supposed to be general enough for usage within other types of `Input Settlers`.

Cantina Managed: Acknowledged.

3.5.11 The string `MandateOutput(...` is repeated

Severity: Informational

Context: MandateOutputType.sol#L22, MandateOutputType.sol#L29-L30, StandardOrderType.sol#L53-L59

Description: The string `MandateOutput(bytes32 oracle,bytes32 filler,uint256 chainId,bytes32 token,uint256 amount,bytes32 recipient,bytes call,bytes context` is repeated in several parts of the code. This make the code more difficult to read and maintain.

Recommendation: Consider creating a `MANDATE_OUTPUT_TYPE_PARTIAL` string and reference this from the other locations.

LI.FI: The recommendation would make the gas costs ~2000-3000 gas higher.

Cantina Managed: Acknowledged.

3.5.12 Order-validation documentation is incomplete

Severity: Informational

Context: GovernanceFee.sol#L69, MandateOutputEncodingLib.sol#L113-L117, MandateOutputEncodingLib.sol#L113-L128, BitcoinOracle.sol#L557, BitcoinOracle.sol#L585-L605, ComponentLib.sol#L185, ComponentLib.sol#L236, DirectDepositLogic.sol#L58, DirectDepositLogic.sol#L117

Description: Several checks / requirements are not part of the order-validation documentation.

Recommendation: Consider adding the following requirement to the documents. Where possible also add these to `RegisterIntentLib::depositAndRegisterIntentFor()`:

- Check all bitcoin fields are filled correctly/consistently;
- Check for an imminent governance fee update;
- Check `call.length` and `context.length` are maximally 65'535 bytes;
- Relation between `output.oracle` and `output.settler` is correct.

The current document says: `Ensure that every input token uses the same lockid.`, this should be changed to:

- Ensure that every input token uses the same `AllocatorId`;
- If the lockIds are different, then the expiry for the lock should be checked for all inputs.

For `BitcoinOracle`:

- When calling `dispute()`, make sure the `claimedOrder.multiplier` is not too high compared to the bitcoin value.

LI.FI: Fixed in orderflow/#order-validation.

Cantina Managed: Fix verified.

3.5.13 Risks for callback functions

Severity: Informational

Context: BaseInputSettler.sol#L142, BaseInputSettler.sol#L195, InputSettlerCompact.sol#L177, InputSettlerCompact.sol#L196, InputSettlerCompact.sol#L237, BaseOutputSettler.sol#L65, BaseOutputSettler.sol#L93

Description: The input and output settlers use callbacks. Callbacks are frequently authorized based on the `msg.sender`. However for the currently implementation this is not sufficient for authorization because

anyone can trigger any callback with any parameters. This is due to the fact that the calling functions `InputSettlerCompact::InputSettlerCompact()`, `InputSettlerCompact::finaliseWithSignature()`, `InputSettlerCompact::purchaseOrder()` and `BaseOutputSettler::fill()` are not authorized.

Recommendation: Consider documenting how callbacks should be authorized.

LI.FI: Acknowledged.

Cantina Managed: Acknowledged.

3.5.14 Emit in function `_fill()` could be expanded

Severity: Informational

Context: BaseOutputSettler.sol#L65-L97

Description: Function `_fill()` send `outputAmount` tokens to the `recipient`, which could be less than `output`. The emit `OutputFilled()` only includes `output`, which makes it difficult to infer the `outputAmount`.

Recommendation: Consider also emitting `outputAmount` from emit `OutputFilled()`.

LI.FI: Fixed in PR 34.

Cantina Managed: Fix verified.

3.5.15 Function `fillOrderOutputs()` reverts when `outputs[]` is empty

Severity: Informational

Context: BaseOutputSettler.sol#L136-L151

Description: Function `fillOrderOutputs()` will revert when `outputs[]` is empty, without a useful error message.

Recommendation: Consider checking `outputs.length > 0` and reverting with a readable error message.

LI.FI: Won't fix. Not a meaningful change. It would be relatively straight forward to debug without the error.

Cantina Managed: Acknowledged.

3.5.16 Typecast to `bytes32()` done twice

Severity: Informational

Context: OutputSettlerCoin.sol#L82, OutputSettlerCoin.sol#L94

Description: Several comments do a typecast to `bytes32()` twice, which is redundant.

Recommendation: Consider changing the comment to:

```
- bytes32(bytes32(output.context[1:33]));
+ bytes32(output.context[1:33]);
```

LI.FI: Fixed in PR 22.

Cantina Managed: Fix verified.

3.5.17 `encodeMessage()` and `decodeMessage()` not symmetric

Severity: Informational

Context: MessageEncodingLib.sol#L31-L33, MessageEncodingLib.sol#L52-L54

Description: The function `encodeMessage()` and `decodeMessage()` are not symmetric although their function names seem to suggest this. This could be confusing to readers and maintainers of the code.

Recommendation: Consider changing the name of `decodeMessage()` to indicate it returns the hashes instead of the messages.

LI.FI: Fixed in PR 21.

Cantina Managed: Fix verified.

3.5.18 Incorrect comment in function `_getProofPeriod()`

Severity: Informational

Context: BitcoinOracle.sol#L108-L112

Description: A comment of function `_getProofPeriod()` says:

```
/**
 * @dev confirmations == 0 returns 181 minutes.
 */
```

However with that input, the function returns `THREE_CONFIRMATIONS + LEAD_TIME` = 119 minutes, which is different.

Recommendation: Consider changing the comment to:

```
- * @dev confirmations == 0 returns 181 minutes.
+ * @dev confirmations == 0 returns 119 minutes.
```

LI.FI: Fixed in PR 20.

Cantina Managed: Fix verified.

3.5.19 `BitcoinOracle::_outputIdentifier()` could use `MandateOutputEncoding::getMandateOutputHash()`

Severity: Informational

Context: MandateOutputEncodingLib.sol#L98-L102, BitcoinOracle.sol#L176-L180

Description: Function `BitcoinOracle::_outputIdentifier()` calls `keccak256(MandateOutputEncodingLib.encodeMand` but it could also directly call `MandateOutputEncoding::getMandateOutputHash()`, which is simpler.

Recommendation:

```
  function _outputIdentifier(MandateOutput calldata output) internal pure returns (bytes32) {
-     return keccak256(MandateOutputEncodingLib.encodeMandateOutput(output));
+     return MandateOutputEncodingLib.getMandateOutputHash(output);
  }
```

LI.FI: Fixed in PR 19.

Cantina Managed: Fix verified.

3.5.20 Code duplication of `_isPayloadValid()` and `arePayloadsValid()`

Severity: Informational

Context: BaseOracle.sol#L36-L43, BitcoinOracle.sol#L288-L308, BaseOutputSettler.sol#L176-L196

Description: `_isPayloadValid()` and `arePayloadsValid()` are duplicated between `BitcoinOracle` and `BaseOutputSettler`. This makes the code more difficult to maintain. Additionally `_isPayloadValid()` directly accesses `_attestations[][][][]` which also makes the code more difficult to maintain.

Recommendation: Consider consolidating the functions `_isPayloadValid()` and `arePayloadsValid()`, for example in `BaseOracle`. Also consider calling `_isProven()` from `_isPayloadValid()` to hide the implementation details of `_attestations[][][][]`.

LI.FI: Not applicable anymore due to changes.

Cantina Managed: Acknowledged.

3.5.21 Several functions write to `_attestations[][][][]`

Severity: Informational

Context: BaseOracle.sol#L21-L24, BitcoinOracle.sol#L406-L407, BitcoinOracle.sol#L624-L625, PolymerOracle.sol#L58, WormholeOracle.sol#L90

Description: Several functions outside of `BaseOracle` directly write to array `_attestations[][][][]`. This make the code more difficult to maintain.

Recommendation: Consider addition a function to update the array `_attestations[][][][]`, to hide the implementation details.

LI.FI: Won't fix. Does not add meaningful abstraction right now.

Cantina Managed: Acknowledged.

3.5.22 `uint32` timestamps will fail beyond the year 2106

Severity: Informational

Context: InputSettlerCompact.sol#L180

Description: Throughout the codebase, we use the `uint32` type to contain timestamps. The maximum value a `uint32` can contain (`type(uint32).max`) is `4294967295`, which represents the date and time: `Sun Feb 07 2106 06:28:15 GMT+0000`. Thus in the year 2106, the protocol will not be able to operate with valid timestamps.

Recommendation: Consider using a `uint40` to represent timestamps, consuming only an additional 8 bits of storage but allowing timestamps to be valid until the year 36786.

LI.FI: Won't fix. Using `uint32` for timestamps is fairly standardised.

Cantina Managed: Acknowledged.

3.5.23 Incorrect `RegisterIntentLib` NatSpec comment

Severity: Informational

Context: RegisterIntentLib.sol#L12-L17

Description: In `RegisterIntentLib.sol`, we include the following NatSpec comment above the library:

```
/**
 * @notice Governance fee timelock
 * Allows for safely setting and changing a governance fee through a built in time-lock. Also provides a
 ↪  generic
 * function to compute the the impact of the governance fee on an amount.
 */
library RegisterIntentLib {
```

This is the same comment used in `GovernanceFee.sol` and should be replaced with a comment describing the `RegisterIntentLib` instead.

Recommendation: Replace the NatSpec comment with one which describes the `RegisterIntentLib`.

LI.FI: Fixed in PR 106.

Cantina Managed: Fix verified.

3.5.24 Order of checks in function `dispute()` not logical

Severity: Informational

Context: BitcoinOracle.sol#L585-L590, BitcoinOracle.sol#L612-L617

Description: Function `dispute()` first checks `claimedOrder.claimTimestamp` and then for `solver !=0` (e.g. the record isn't empty). It seems more logical to do this the other way around, similar to function `optimisticallyVerify()`.

Recommendation: Consider changing the code to:

```
+ if (claimedOrder.solver == bytes32(0)) revert NotClaimed();
  if (claimedOrder.claimTimestamp + DISPUTE_PERIOD < block.timestamp) revert TooLate();
- if (claimedOrder.solver == bytes32(0)) revert NotClaimed();
```

LI.FI: Fixed in PR 18.

Cantina Managed: Fix verified.

3.5.25 `DISPUTE_PERIOD` can be shorter than the number of confirmations requested

Severity: Informational

Context: BitcoinOracle.sol#L612-L617

Description: The `DISPUTE_PERIOD` (4 confirmations) can be shorter than the number of confirmations requested in `_getNumConfirmations(token)`, which could be 255. This might be unexpected.

Recommendation: Doublecheck if this it the expected behavious, if not change the code.

LI.FI: Won't fix. It is assumed that transactions will get disputed fairly quickly after the inclusion window. This means disputers have roughly 2 blocks to dispute transactions in before they can be optimistically verified.

Cantina Managed: Acknowledged.

3.5.26 Only function for one direction of mapping in `ChainMap`

Severity: Informational

Context: ChainMap.sol#L34-L39

Description: There is only a function for one direction of mapping in `ChainMap`, e.g. `_getMappedChainId()`. It might be useful to also have a similar function to convert `chainid` to `protocolId`.

Recommendation: Consider adding a function to convert `chainid` to `protocolId`.

LI.FI: Won't fix. Code would currently be unused.

Cantina Managed: Acknowledged.